

10 Prompts I Use Every Day

Battle-tested patterns from shipping production apps with Claude Code daily. Not theory — the actual prompts I reach for.

By @koki7o • thedevfounder.com

March 2026

1 The "Onboard Me"

When: Opening unfamiliar or forgotten code

```
Explore this codebase and give me:
1. What it does (one paragraph)
2. Tech stack
3. Entry point
4. How to run it
```

→ Oriented in 30 seconds, not 30 minutes

2 The "Do It Like the Rest"

When: Adding features to existing code

```
Add [feature]. Look at how [existing feature]
is implemented and follow the exact same
patterns – structure, naming, error handling.
```

→ Consistency > cleverness

3 The "Fix This Error"

When: Something broke

```
I'm getting this error:
[paste FULL error + stack trace]
```

Read the files, explain it, fix it, run tests.

→ Full stack trace = file paths + line numbers Claude needs

4 The "Review My Changes"

When: Before committing

```
Review git diff for:
- Bugs or logic errors
- Security issues
- Missing error handling
- Things I forgot
```

Be specific about line numbers.

→ Catches what your eyes miss after hours of coding

5 The "Write Tests"

When: You skipped tests (we all do)

```
Write tests for [file] covering:
- Happy path (normal usage)
- Edge cases (empty, null, bounds)
- Error cases (what should fail)
```

Follow existing test patterns.

→ 3 categories = comprehensive, not just happy-path

6 The "Explain This"

When: Reading code you don't understand

```
Explain [function/file/module].
Walk through step by step as if I know
the language but never saw this codebase.
```

→ "Know the language" prevents over-explaining syntax

7 The "Production Ready"

When: Prototype works, need to ship

```
Review for production readiness:
- Error handling - Security
- Validation - Rate limiting
- Logging - Env vars
```

Fix critical issues. List the rest.

→ Checklist in the prompt = nothing gets skipped

8 The "Safe Refactor"

When: Code works but is ugly/slow

```
Refactor [file] for readability.
Rules:
- Keep behavior identical
- Run tests after each change
- If no tests, write them FIRST
```

→ "Tests FIRST" prevents refactoring untested code

9 The "Full Commit"

When: Done with a chunk of work

```
Look at all changes (git diff).
Create a commit with a clear message:
what changed and why.
```

→ Claude writes better commit messages — it sees the full diff

10 The "Set Up My Project"

When: Starting or joining a project

🌟 Bonus: The Meta-Prompt

When you don't know how to ask:

Create a CLAUDE.md based on:

- Actual tech stack + deps
- Existing coding patterns
- Test setup + build commands

Then create .claude/rules/ for language-specific conventions.

→ Auto-generated from real code beats writing from scratch

I want to [goal]. Not sure the best approach.
Suggest 2-3 approaches, recommend one,
then implement it.

The Pattern Behind Every Good Prompt

Context What to look at — *"read this file", "look at git diff"*

Task What to do — *"fix it", "add tests", "refactor"*

Constraints How to do it — *"follow existing patterns", "run tests after", "keep behavior identical"*

[Context] Read the auth module.

[Task] Add password reset functionality.

[Constraints] Follow existing login/register patterns. Include tests.